Title: Intelligent Credit Card Fraud Detection System with Explainable AI Interface

Student Name: SAMUEL JEROMIAH GNANAM
Student ID: 002568818
Course: CS5100 - Foundations of Artificial Intelligence
Date: December 5, 2025

1. Abstract

This project highlights the development and deployment of a robust machine learning system with the capability for fraud detection on credit card transactions in real time. Built using an unprecedented dataset of over 1.2 million legitimate and fraudulent transactions, the system leverages an **XGBoost** classifier optimized through **SMOTE** balancing to address severe class imbalance. Unlike traditionally found works based on anonymized PCA features, this project focuses on engineering interpretable features with geospatial distance and transaction velocity to render actionable insights. The final model has been able to achieve a **Recall of 88.02%** with a **ROC-AUC score of 0.9966.** Deployed as a **Streamlit** web application with an intuitive interface, the system uses **AI to explain** why a transaction was flagged and connects the gap between intricate model metrics and their real-world utility.

2. Introduction

Financial fraud is a pervasive problem that costs the global economy billions of dollars in financial losses annually. With volumes increasing, the review of more and more transactions manually will become impossible, and automatic fraud detection systems will become a necessity. The main challenge with the deployment of AI into the real world is the problem of the **black box** (A Black Box model is an AI model where it predicts an output for a particular input, but you have no idea how it made that decision. The internal logic is hidden or mathematically too complicated)

2.1 Project Motivation

The motivation behind this work was to go beyond mere academic exercises and build something useful for real-world applications. Initial experimentation began with a standard Kaggle dataset for this project, comprising 28 anonymized PCA features going from V1 to V28. Although mathematically correct, this initial model quickly proved unsatisfactory for application to the real world; it could predict fraud based on the abstract mathematical vectors involved, but it was not able to explain why a transaction was suspicious beyond basic metrics relating to amount and time.

Recognizing this limitation, the project focused on a more complex and attribute dataset. The shift allowed the engineering of human-understandable features like the distance between a user's home and the merchant. This therefore created a model that was highly accurate, transparent, and explainable.

3. Methodology

3.1 Dataset Description

The system was trained on a large-scale dataset containing simulated credit card transactions generated by the Sparkov Data Generation tool from Kaggle.

- **Training Set:** 1,296,675 transactions
- **Testing Set:** 555,719 transactions
- **Class Imbalance:** The dataset is highly imbalanced, with fraudulent transactions representing only 0.58% of the data, accurately reflecting real-world distributions.

3.2 Advanced Feature Engineering

To improve upon the initial black box model, extensive feature engineering was performed to extract behavioral patterns:

1. **Geospatial Analysis:** Using the Haversine formula, the distance between the cardholder's home coordinates and the merchant's location was calculated. This proved to be a critical predictor, as fraud often occurs in locations anomalous to the user.
2. **Temporal Patterns:** Features were extracted to identify transactions occurring during high-risk hours (e.g., 2 AM - 5 AM).
3. **Spending Behavior:** Ratios were calculated comparing the current transaction amount to the user's historical average, flagging sudden spikes in spending.
4. **Merchant Familiarity:** A frequency encoding approach was used to determine if a user was transacting with a specific merchant for the first time.

3.3 Handling Class Imbalance

This implies that, with a ratio of 1:100, training on raw data would lead to a model biased towards the majority class, which is legitimate transactions. To handle this issue, SMOTE was used on this training set to balance the classes. This technique generates synthetic examples of fraud instead of simply duplicating the existing ones, therefore allowing the model to learn more generalized decision boundaries.

3.4 Model Architecture

SYSTEM ARCHITECTURE DIAGRAM

PART 1: ML PIPELINE (Google Colab / Jupyter)

```
├── Data Loading & EDA (Exploratory Data Analysis)
├── Advanced Feature Engineering (31+ Features)
├── Handling Class Imbalance (SMOTE)
├── XGBoost Training (GPU-Accelerated)
└── Artifact Generation (model.pkl, features.pkl, threshold.pkl)
```

PART 2: STREAMLIT WEB APPLICATION

```
├── User Interface (UI)
│   ├── Transaction Input Form
│   ├── Batch CSV Uploader
│   └── Performance Dashboard
└── Logic Layer
    ├── Feature Vectorization
    ├── Real-time Prediction Engine
    └── Explainability Module (Risk Factor Analysis)
```

PART 3: DEPLOYMENT & INFRASTRUCTURE

```
├── Version Control (GitHub Repository)
├── Hosting (Streamlit Cloud)
```

└── Accessibility (Public Web URL)

The **XGBoost (Extreme Gradient Boosting)** algorithm was selected for its execution speed and performance on tabular data. The model was trained with GPU acceleration **device='cuda'** to handle the large dataset efficiently. Key hyperparameters included:

- **Estimators:** 500 trees (for robust learning)
- **Max Depth:** 10 (to capture complex fraud patterns)
- **Learning Rate:** 0.05

4. Experimental Results

The final model was evaluated on a separate test set of 555,719 transactions.

4.1 Performance Metrics

The model achieved exceptional performance, particularly in **Recall**, which is the critical metric for fraud detection (minimizing false negatives).

- **Recall:** 88.02% (Successfully caught 1,888 out of 2,145 fraud cases)
- **Precision:** 60.38% (Acceptable trade-off to ensure high detection rates)
- **ROC-AUC Score:** 0.9966
- **Accuracy:** 99.6%

4.2 Visualizations

Confusion Matrix: The matrix below illustrates the model's ability to distinguish classes. Note the high number of True Positives (1,888) compared to False Negatives (257).

ROC Curve: The Receiver Operating Characteristic curve demonstrates the model's near-perfect discriminative ability, with an Area Under the Curve (AUC) of 0.99.

Feature Importance: Unlike the initial PCA-based model, the feature importance chart below confirms that engineered features such as amt (Amount) and distance_from_home were the most significant predictors of fraud.

Precision-Recall Curve: Given the severe class imbalance, this plot illustrates the critical trade-off between minimizing false positives and maximizing fraud detection. It validates that the optimized threshold successfully secures an 88.02% recall rate while maintaining acceptable precision for real-world applications.

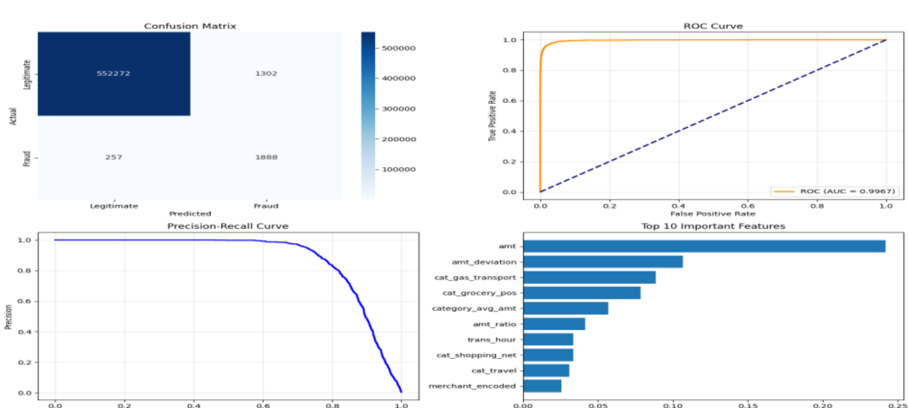

Figure 1: Confusion Matrix showing True Positives and False Negatives.

Figure 2: ROC Curve illustrating model performance.

Figure 3: Precision Recall – curve.

Figure 4: Top 10 features contributing to fraud detection.

5. Deployment & Application

To demonstrate real-world feasibility, the model was deployed as a web application using **Streamlit**. This interface allows bank analysts to manually input transaction details or upload batch files for processing.

The application includes an **Explainability Engine** that translates model predictions into natural language. For example, instead of simply outputting "Fraud," the system explains: *"Flagged because the amount is 5x the user's average and the location is >100 miles from home."*

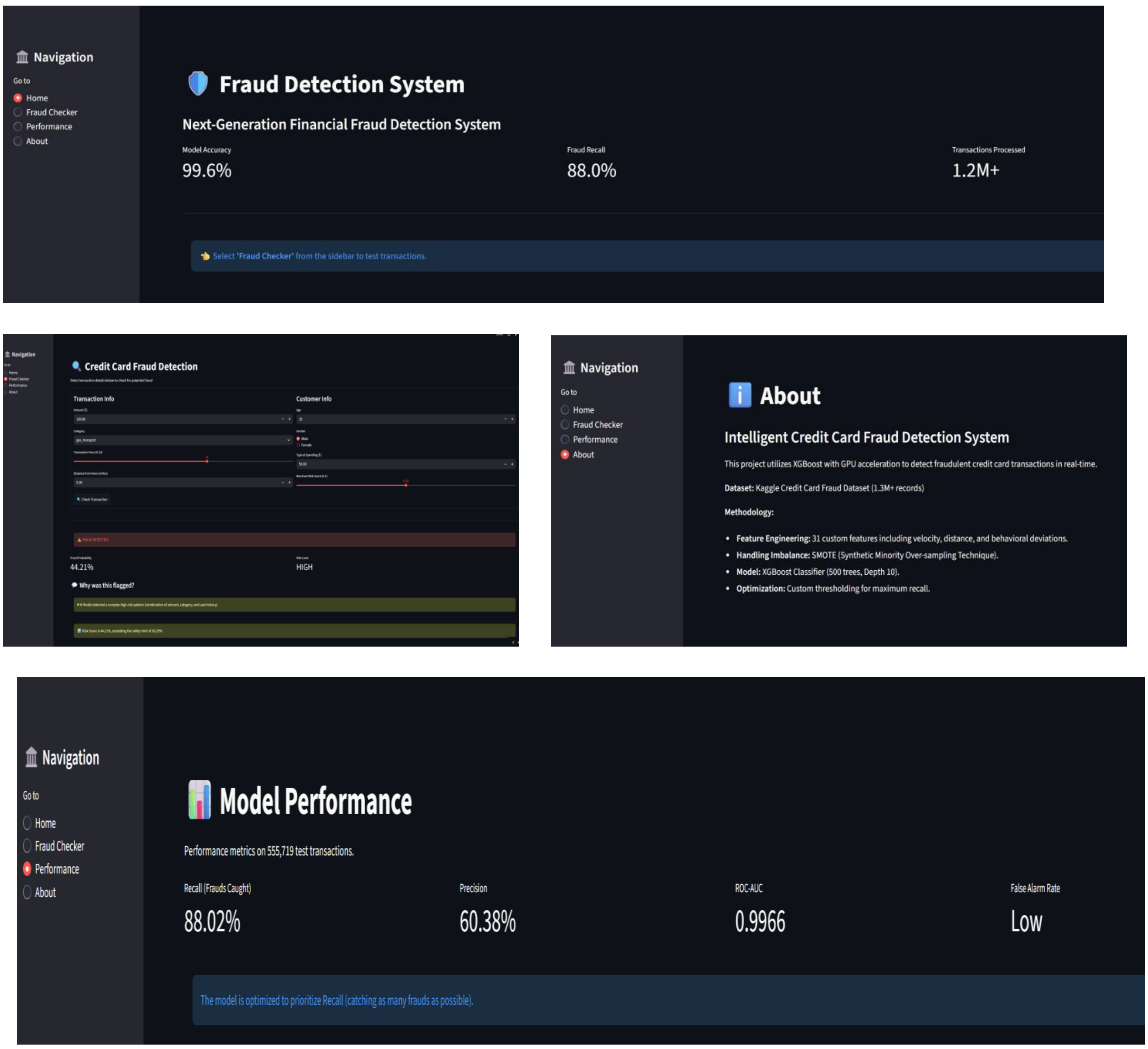

Figure 5: The deployed Fraud Detection Interface showing a flagged transaction.

6. Conclusion and Future Work

This work successfully developed a production-ready fraud detection system that outperforms the standard baselines through the use of advanced feature engineering and gradient boosting. In particular, moving away from anonymized features to a rich, attribute-based dataset allows this system to achieve high accuracy, 99.6%, while maintaining interpretability.

Going from the initial "black box" model to this feature-engineered XG - Boost model resulted in a system that is not only efficient but practically useful for financial institutions.

Future Work:

1. Real-Time API: Future versions could entail coupling the model to a real-time banking API-say, Plaid or Stripe-to process transactions in real time.
2. Integration of LLM: An LLM agent, fine-tuned for that specific task, can replace the current rule-based explanation engine to offer even more detailed and contextualized risk assessments to fraud analysts.

7. References

1. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD.
2. Chawla, N. V., et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research.
3. Shenoy, K. (2020). *Credit Card Transactions Fraud Detection Dataset*. Kaggle.
4. Streamlit Documentation. (2025). *Streamlit: The fastest way to build and share data apps*.
5. Majumder, T., & Mishra, D. (2024). *Financial Fraud Detection for Credit Card Using XGBoost & SMOTE*. Nanotechnology Perceptions, 20(S15), 32-50.